Algorithm final

Jannatul Mawa

Greedy Algorithm

① Greedy ② DP ③ Braneh and bound.

↓

(Sub braneh of divide and conquere)

Greedy Algo

↓

Min / max (output)

↓

optimization ~~rang~~ Prob (Maximum Output)

↓

Operational rang ($0 - 10^5 / 10^6$)

* Greedy বড় rang ke সঠিক জন্য কাজ করবে না, TLE দেখাবে।
* DP বড় Rang এর Problem এর জন্য কাজ করে।
* Greedy দিয়ে ২ ধরনের Problem Solve করা যায়

① Knapsack.

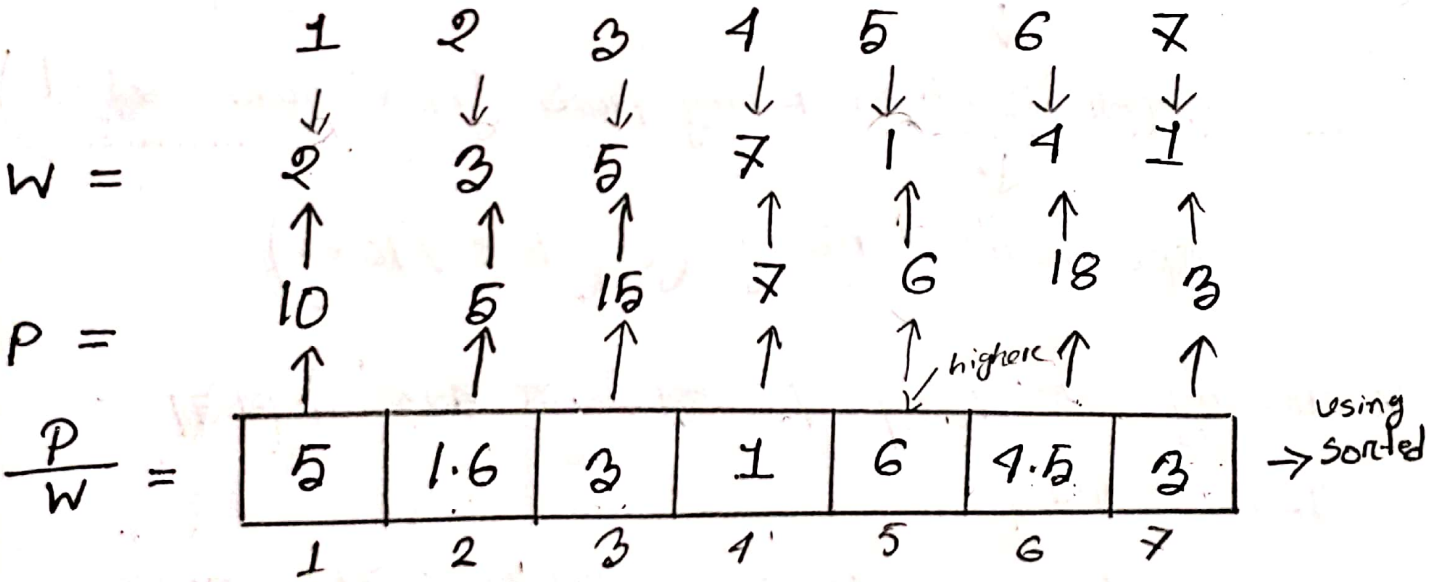
② Job sequence / Activity Selection problem

③ 0/1 knapsack. (Dp Method)

Knapsack (Fractional knp) ✓

Profit Per Weight ⇒ প্রতি kg-তে- কত-

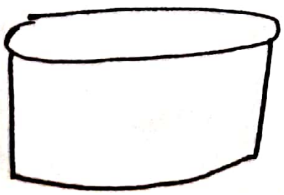
Profit বের করা ($\frac{P}{W}$)



x = x_1 x_2 x_3 x_4 x_5 x_6 x_7

(2) (2) (5) 0 (1) (4) (1)

2nd 6th 4th 1st 3rd 5th



→ 15 kg = C

R.W = Remaing Weight.

$W_1 = 1$	$P_1 = 6$	R.W = 15 - 1 = 14
$W_2 = 2$	$P_2 = 10$	R.W = 14 - 2 = 12
$W_3 = 4$	$P_3 = 18$	R.W = 12 - 4 = 8
$W_4 = 5$	$P_4 = 15$	R.W = 8 - 5 = 3
$W_5 = 1$	$P_5 = 3$	R.W = 3 - 1 = 2
$W_6 = 2$	$P_6 = \frac{2 \times 5}{3}$	R.W = 2 - 2 = 0

Profit = $\sum P_i = 55.33$ [Total Profit]

$\sum W_i = 15$ [total Weigh]

$i = 6$. [Product Num]

Jannatul Maoa

Job Sequencing / Activity Selection

Job sequencing / Activity Selection (deadline) using Greedy Method.

- কোন কাজ কোন time এর মধ্যে শেষ করলে Profit বেশি হয় (Greedy).

5 জ

	J_1	J_2	J_3	J_4	J_5
P	20	15	10	5	1
Deadline	2	2	1	3	3

সর্বোচ্চ Scope

Needtime → 1 Unit of time.

[0 → 1 → 2 → 3]

Sorted করা লাগবে বড় থেকে ছোট।

$$0 \xrightarrow{J_2\ (2nd)} 1 \xrightarrow{J_1\ (1st)} 2 \xrightarrow{J_4\ (3rd)} 3$$

$J_2 + J_1 + J_4$

$\Rightarrow 20 + 15 + 5$

$\rightarrow 40.$

Jannatul Mawa

Job	Slot	Sequence	Profit
J_1	[1,2]	J_1	20
J_2	[0,1] [1,2]	$J_2 J_1$	35
J_3	[0,1] [1,2]	$J_2 J_1$	35
J_4	[0,1] [1,2] [2,3]	$J_2 J_1 J_3$	40
J_5	[0,1] [1,2] [2,3]	$J_2 J_1 J_3$	40

Higest = 40 Job sequence = $J_2 J_1 J_3$.

✓ Halfman Coding (Optimal Marge Pattern)

wp, Email, massanger

(20) Sending file capacity (25 MB) — wp

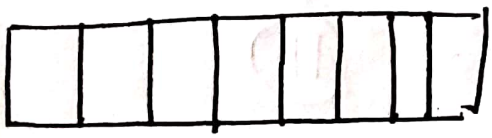
120 MB → (25)

যদি 120 MB এর কোনো file পাঠানো হয় সেটা convert হয়ে 25 MB হয়ে যায়

[8bit - 1 byte]

BCC A BB BA ....

B → 66, C → 67, A → 65



— 8 bit —

→ 8X20
= 160 bit

[Constant Size data formating]

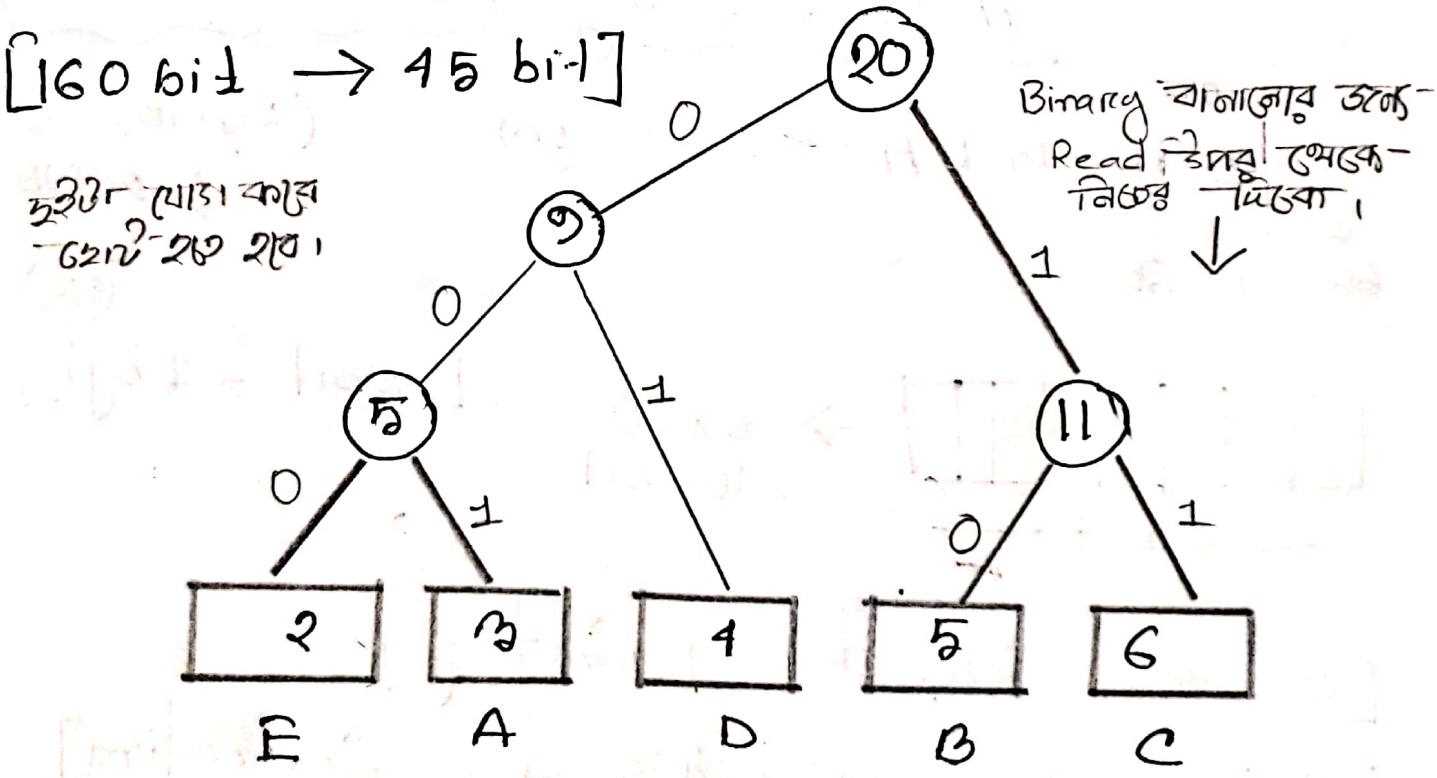
[Halfman → Veriable Size data formating]

যদি কোনো Pic/video 125 MB এর হয় তবে সেটি Document করে Send করলে সেটা zip file এর মতো কাজ করে তখন 125 থেকে 25 MB হবে কিন্তু [illegible] Download এর পর 70-80 MB হয়ে যাবে।

B C C A B B D D A E ---- 20

Char	Count	Code	Size	
A	৩	001	3×3	৯
B	৫	10	5×2	10
C	6	011	6×2	12
D	৪	01	4×2	৪
E	২	000	2×৩	6
	20	12 bit		45 bit

[160 bit → 45 bit]



Message = 45 bits (char, code)

Key = (8×5) + 12 = 52 bits

Minimum Spaning tree

Seq-5

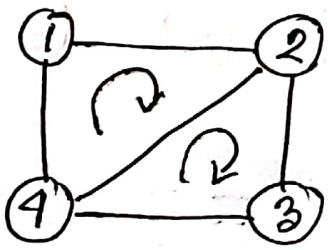
Non directed graph (weighted)

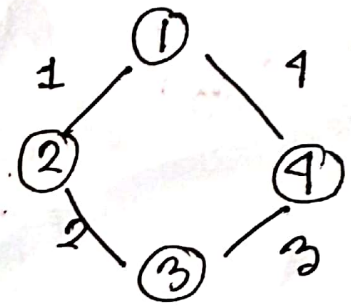
Graph → Minimum Spaning tree.

(set of vertex and edges) (optimization)

Mini

Rules = $[\ E C_{E-1}\ ] = E^2$





→ (E-I)

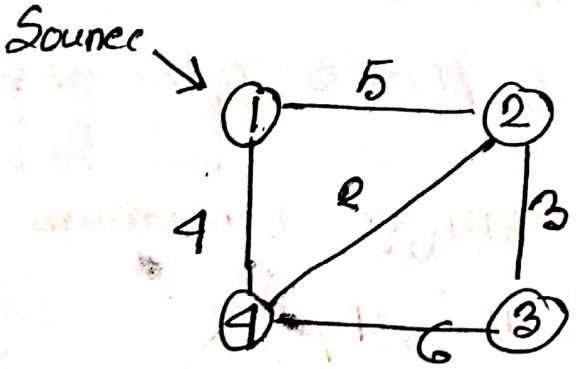
↓

(Spaning tree)

$G_1(V, E) = G_1(4,4)$

$V = \{1, 2, 3, 4\}$

$E = \{(1,2)\ (2,3)\ (2,4)\ (1,4)\}$,

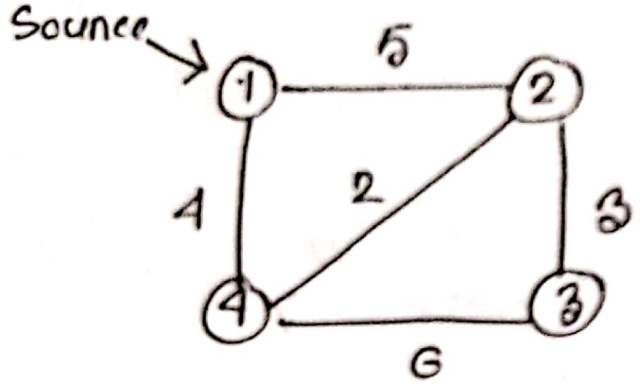


(Minimum cost Spaning tree)

Jannatul Macoa

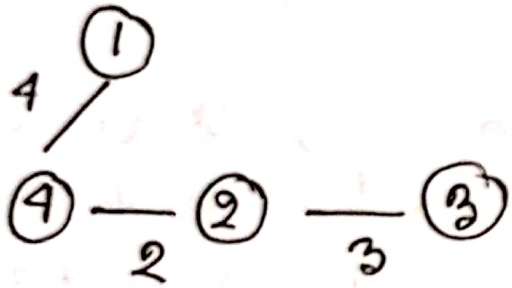
Lucazol™

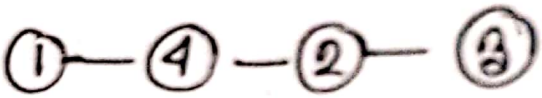
Prim's Algorithm — Svg-3



* Greedy Method

[Node অনুযায়ী -]

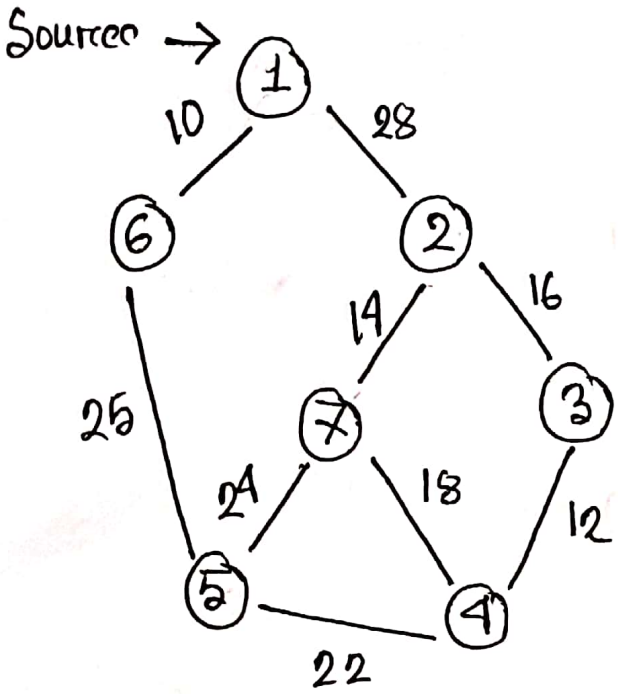




cost : 9.

- Prim's and kruskal algo undirected graph -এর জন্য প্রযোজ্য।
- Minimum spanning tree বের করার - Process দুটি। যেমন: (i) Prim's (ii) kruskal.
- Single Source -এর সাপেক্ষে Minimum Spaning tree বের করা হয়।
- Start Node থেকে- minimum value নিয়ে কাজ করবে।
- Sleeted Node গুলোর সাথে এর সাথে Adjeet Vertex গুলোর মধ্যে- minimum Vertex নিয়ে কাজ করবে

Kruskal Algorith - Sag-5



* Greed Method

(Sorted edges Vertex formet)

(ciele detect) → 3ই vertex কাজ হবেনা।

Usual Time Complexity

$O(n^2)$

Min heap Apply:

$O(n \log n)$

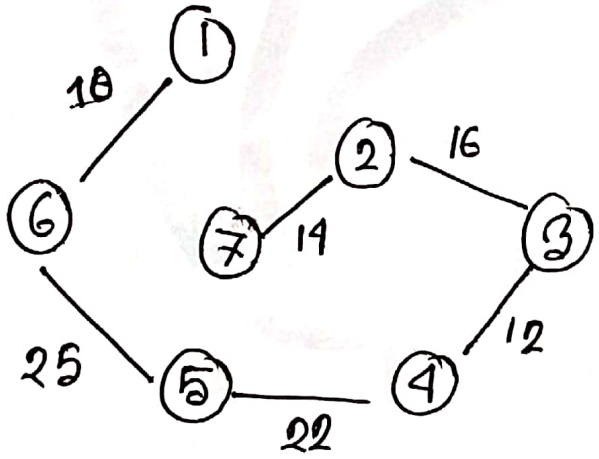
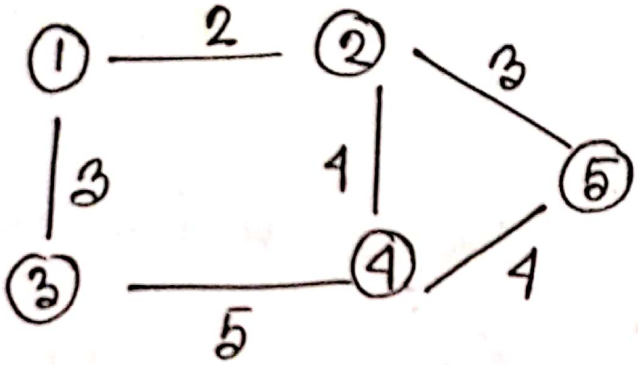
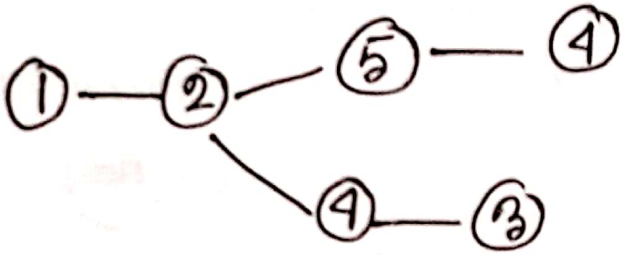


Fig. Saving tree.

Jannatul Mawa

Ex-2





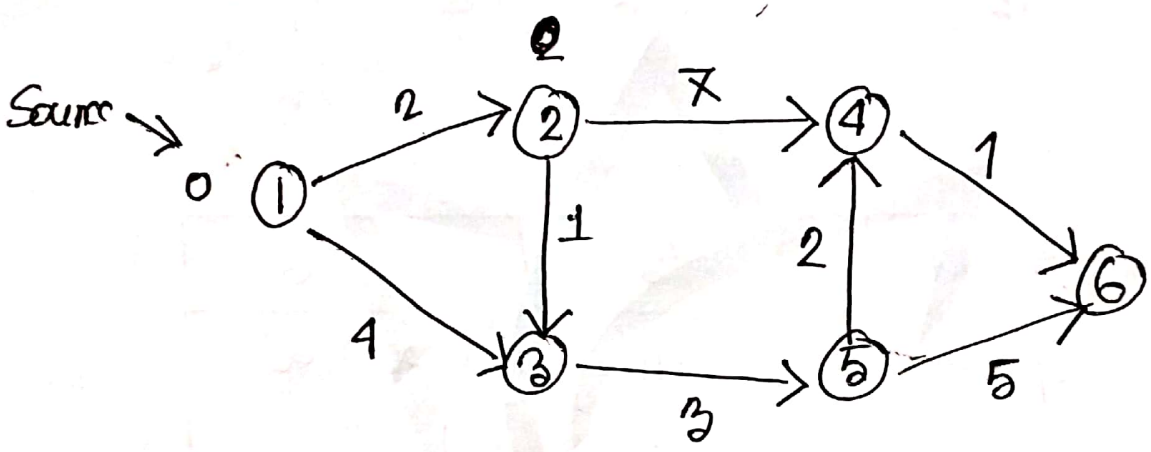
Dijkstra Algorithm (ডাইজস্ট্রা)

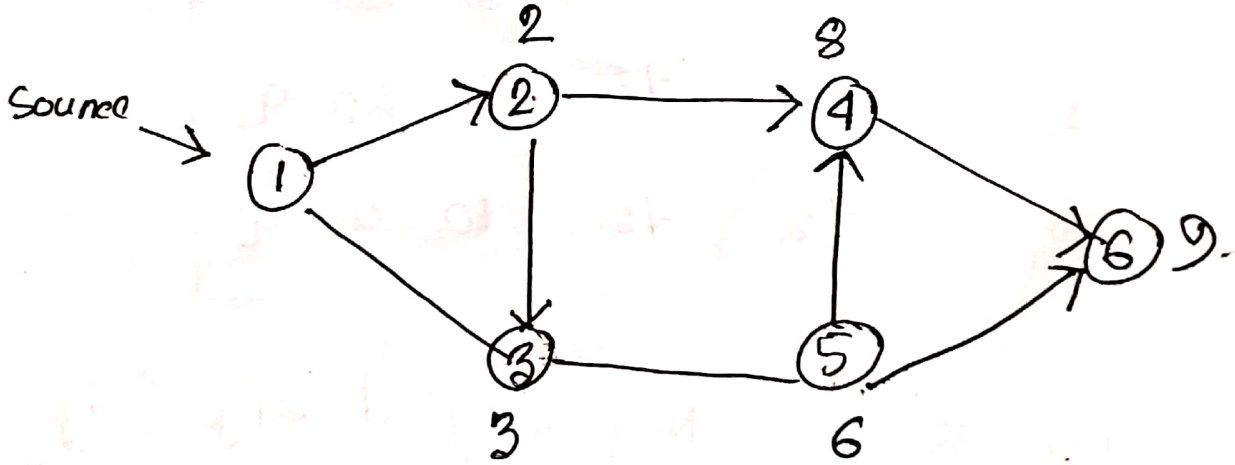
Single Source Shortest path * Greedy Method

Relaxation:- [if (d[u] + c(u,v) < d[v] }

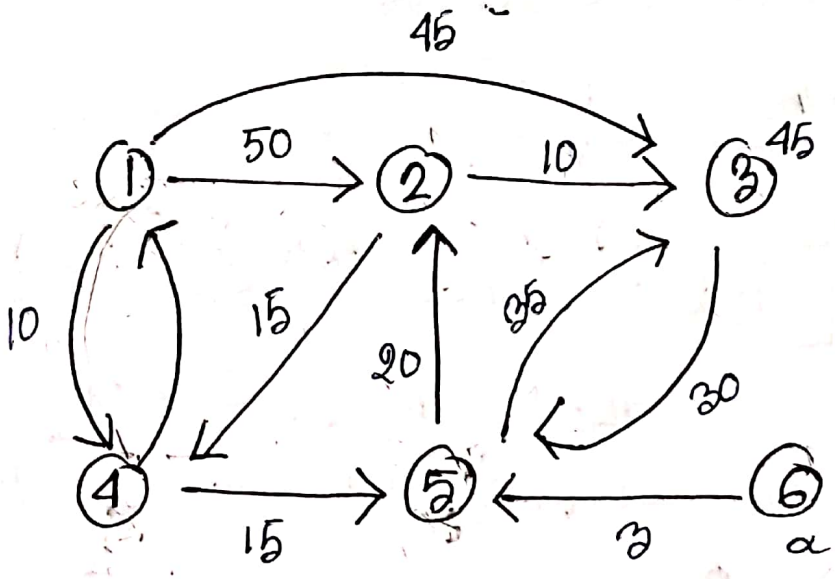
d[v] = d[u] + c(u,v)]

* Directed ও No directed দুইটির জন্য কাজ করে।





[একটি নির্দিষ্ট Source থাকে, সেটা থেকে সবগুলো Node কম খরচে কিভাবে যাওয়া যায় সেই Graph এর বাকি Node কম খরচে কম সময়ে কোন পথ অতিক্রম করতে হবে।



Selected Vertex	2	3	4	5	6
4	50	45	(10)	∞	∞
5	50	45	(10)	(25)	∞
2	(45)	45	(10)	(25)	∞
3	(45)	(45)	(10)	(25)	∞

* যদি Graph এর মধ্যে Neg weighted edge (edge এর value যদি (-) ধনাত্মক হয়) সে Graph এর জন্য Dijkstra algo কাজ করবে না।

* Neg যুক্ত Graph এ Bellmen-Ford's Algo

Bellman ford's

Jannatul Macoa

- Dynamic Programming Method-এ কাজ করে।
- Single Source Sortest Path.
- Neg-edge এর সমস্যা সমাধান করে। কারণ Dijkstar algorithm neg-edge এর সমস্যা সমাধান করতে পারে না। So, Bellma ford's Dijkstar এর Solution হিসেবে কাজ করে।
- আবার Bellman ford's graph এর মধ্যে cycle থাকে এবং cycle এর যোগফল যদি Neg-হয় সেক্ষেত্রে Bellman-ford কাজ করবে না। এর Solution এর জন্য Warshall's Algo ব্যবহার করা হয়।
- আমরা বলতে পারি, Condition: from Graph:-
  1. সকল Positive edges এর Dijkstar algo
  2. Non cycle and neg edge এর জন্য Bellman-ford's.
  3.

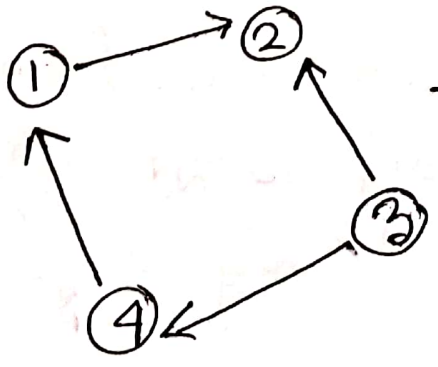
For Single Source Sortest Path:

একটি Source থেকে কম খরচে আলাদা আলাদা সেই Vertex অতিক্রম করাকে Single Source Sortest Path বলে।

- আমরা জানি, Bellman-fords Algo এর SSSP Negetive edge এর জন্য কাজ করে।

Suppose, Vertex $\longrightarrow n$

প্রতি Vertex এর Relaxation operation হলো $= (n-1)$ time.



→ * It has, node = 4
* Relaxation Op = 4-1 = 3
* Non cyclic graph.

- Bellman Ford's Algo Directed, Non directed উভয়ের জন্য কাজ করে।

Jamatul Maaa9

Nod n = 7

R.O = n-1
= 6

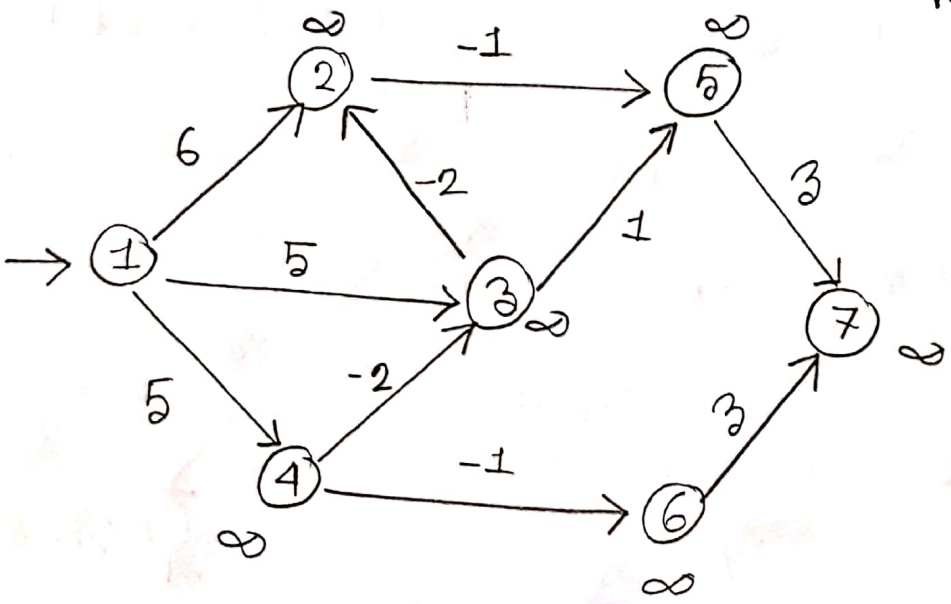
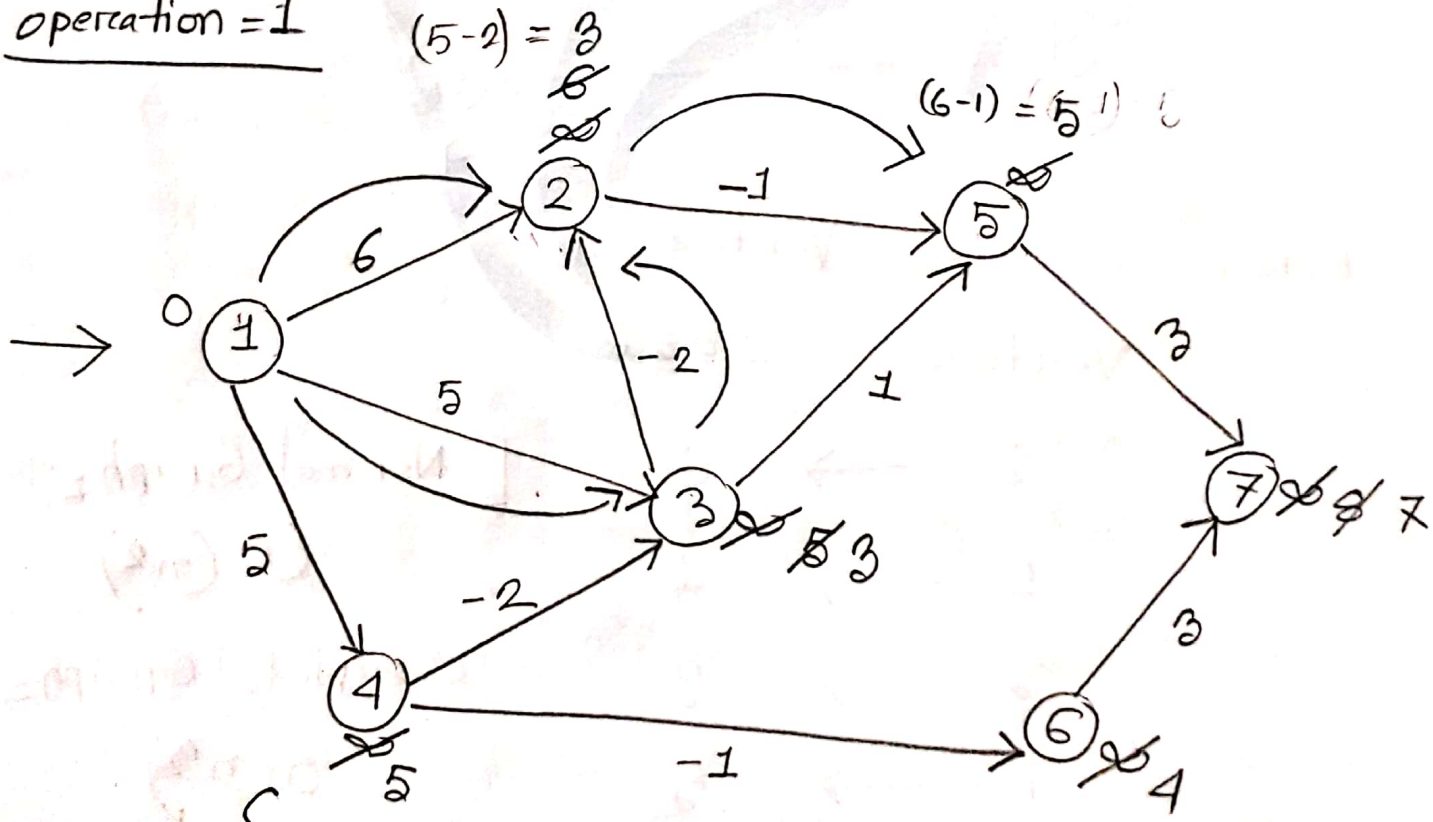


fig.: Neg Weighted Graph. (Directed)

operation = 1

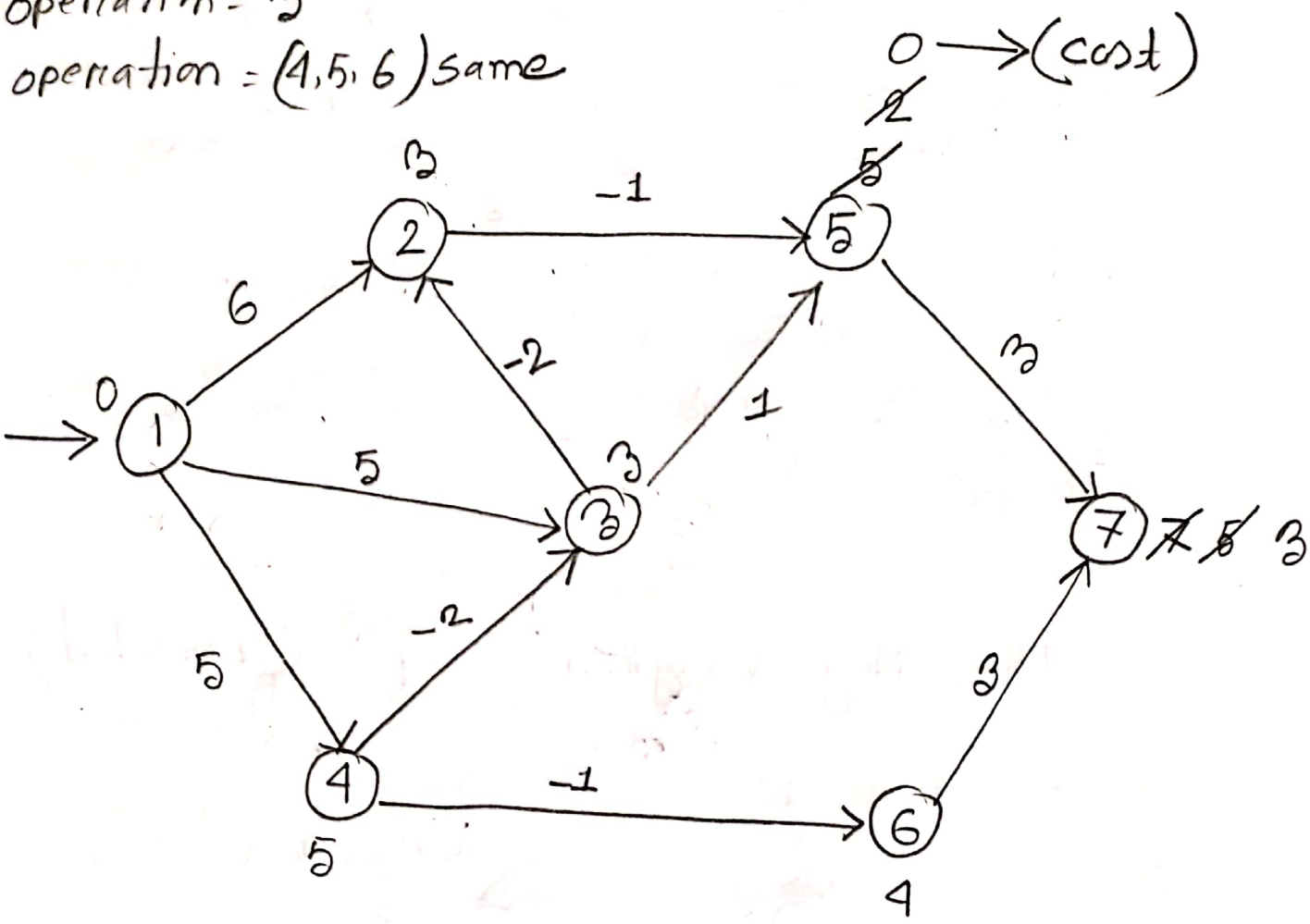


Edges: {(1,2)(1,3)(1,4)(2,5)(3,2)
(3,5)(4,3)(4,6)(5,7)(6,7)}

operation = 2

operation = 3

operation = (4, 5, 6) same



Here, Source Vertex = 1

Vertex	cost
1 →	0
2 →	1
3 →	3
4 →	5
5 →	0
6 →	4
7 →	3

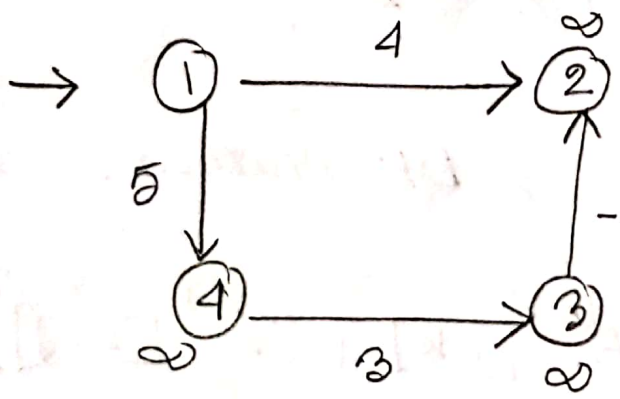
[Normal Graph = $O(n^2)$

Complete Graph = $O(n^3)$

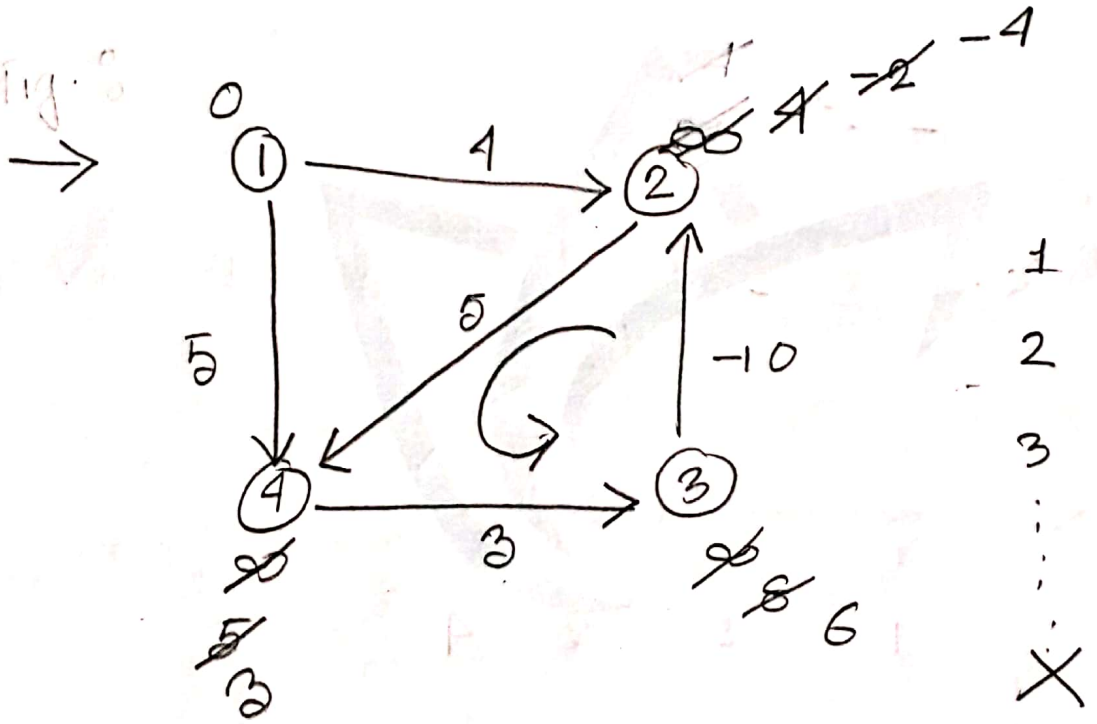
* Comple Graph হলো যেখানে প্রতিটি Node থেকে প্রতিটি Node সরাসরি Connected.]

Fail of Bellman ford's

Jamalul Nasar



→ Not cyclalic



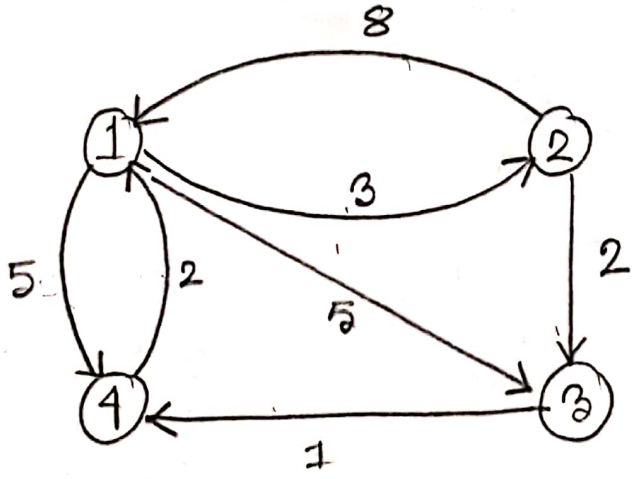
সুতরাং; Cyclalic graph যার আওতাভুক্ত নয় সেটা কখনোই Bellman ford's algo কাজ করবে না। কারণ সেই cycle-এ continously cost chang হতে থাকবে। এবং এ process শেষ হবে না Relxation operation-এর Limitation - অনুযায়ী।

All paire Shortest Path

(Floyed Warshall's)

All paire Shortest Path → DP Mathod.

$$A^k[i,j] = \min\{A^{k-1}[i,j], A^{k-1}[i,k] + A^{k-1}[k,j]\}$$



$$A^0 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{bmatrix} \end{array} \rightarrow \text{Matrix}$$

Jannatul Mowa

$$A^1 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{array}$$

[দুইটি Node 1 এর মাধ্যমে connect হচ্ছে]

[1st row 1st column সেই থাকবে]

(2-1-3) মানে হলো 2 ও 3 কে connect করব 1 দিয়ে

$$A^K[i,j] = \min\{ A^{k-1}[i,j], A^{k-1}[i,k] + A^{k-1}[k,j] \}$$

$$A^1[2,3] = \min A^0[2,3], A^0[2,1] + A^0[1,3]$$

$$= \min(2, 8+\infty)$$

$$= \min(2, \infty)$$

$$= 2$$

$$A^1[2,4] = \min(A^1[2,4], A^0[2,1] + A^0[1,4]$$

$$= \min(\infty, 8+7)$$

$$= \min(\infty, 15)$$

$$= 15$$

$$A^1[3,2] = \min(A^1[3,2],\ A^0[3,1] + A^0[1,2])$$

$$= \infty,\ 5+3$$

$$= 8$$

$$A^1[3,4] = \min(A^0[3,4],\ A^0[3,1] + A^0[1,4]$$

$$= 1,\ 5+7$$

$$= 1$$

$$A^1[4,2] = \min(A^1[4,2],\ A^0[4,1] + A^0[1,2])$$

$$= \min(\infty,\ 2+3)$$

$$= \infty, 5$$

$$= 5$$

$$A^1[4,3] = \min(A^0[4,3],\ A^0[4,1] + A^0[1,3])$$

$$= \infty,\ 2+\infty$$

$$= \infty$$

Jannatul Maoua

$$A^2 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & (5) & (7) \\ 8 & 0 & 2 & 15 \\ (5) & 8 & 0 & (1) \\ (2) & 5 & (7) & 0 \end{bmatrix} \end{array}$$

[2নং row column Same থাকবে]

$$A^2[1,3] = \min\{A^{k-1}[1,3], A^1[1,2] + A^1[2,3]\}$$

$$= \min(\infty, 3+2)$$

$$= 5$$

$$A^2[1,4] = \min\{A^1[1,4], A^1[1,2] + A^1[2,4]\}$$

$$= \min(7, 3+15)$$

$$= 7.$$

Lucazol™

A^3 =

	1	2	3	4
1	0	(3)	5	(6)
2	(7)	0	2	(3)
3	5	8	0	1
4	(2)	(5)	7	0

[3rd row 3rd column same]

shortcut ⇒ [1,2] [1 3 + 3 2]

3 < 5 8

⇒ [1,4] [1 3] + [3 4]

7 5 1

7 > 6

= [2,1] [2 3] + [3 1]

= 8 2 5

= 8 > 7

= [2,4] [2,3 + 3,4]

= 15 > = 2/3 + 1

Jannatul Maoua

$[4,1] \quad [4\,3+3\,1]$

$2 < 7+5$

$[4,2] \quad [4\,3+3\,2]$

$5 < 7+8$

$$A^4 = \begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 3 & 5 & 6 \\ 5 & 0 & 2 & 3 \\ 3 & 6 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{bmatrix} \end{array}$$

$[1,2] \quad [1,4+4,2]$

$3 < 6+5$

$[1,3] \quad [1,4+4,3]$

$5 < 6 \quad 7$

$[2,1] \quad [2,4+4\,1]$

$7 > 3+2$

$[2,3] \quad [2,4+4,3]$

$2 \quad 3 \quad 7$

$[3,1] \quad 3,4+4,1$

$5 \quad 1 \quad 2$

$5 > 3$

$[3,2] \quad 3,4,4\,2$

$8 \quad 1+5$

- All paire Shortest Path Dijsktra Pro version. যেহেতু এতে Neg edge নাই - সেহেতু Dijaskstra Extended Version use হয়, Dijsktra এর n time use হয় যেখানে n টা source থাকে এবং source = node Num.

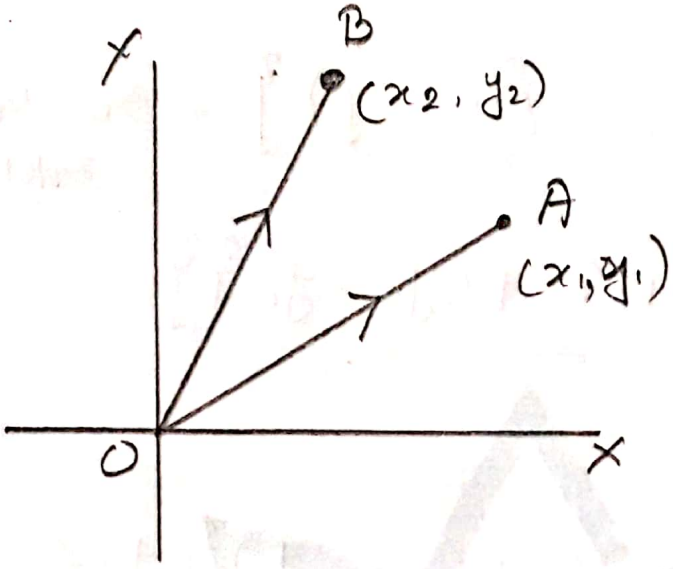
- প্রতিটি Node থেকে প্রতিটি Node এ যাওয়ার ন্যূনতম Path বের করার Process হলো - Floyed Warshall.

- Time Compliexity = $O(n^3)$.

Computational Geometry

Jannatul Maowa

* Vector : যার মান দিক দুইটাই আছে।

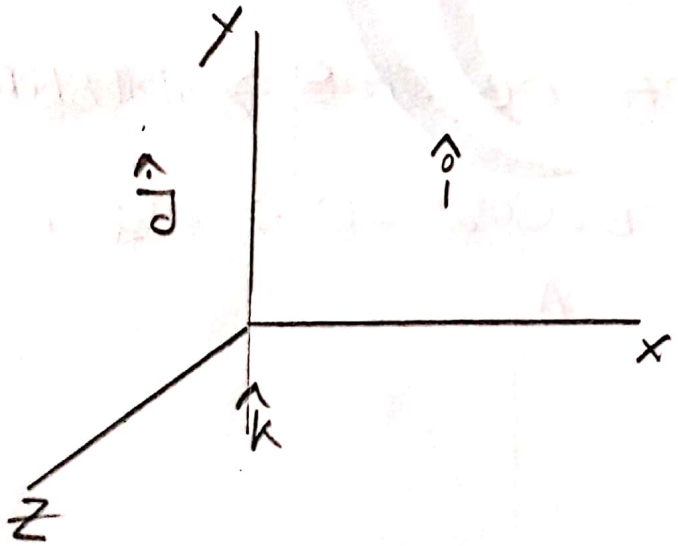


$\vec{OA} = x_1\hat{i} + y_1\hat{j}$

$\vec{OB} = x_2\hat{i} + y_2\hat{j}$

A বিন্দুর অবস্থানাবদ্ধ মান দ্বারা Vector প্রকাশ করা,

$A - O ; (x_1\hat{i} + y_1\hat{j})$



মূলবিন্দু O-র সাপেক্ষে A বিন্দুর Connection কে অবস্থান ভেক্টর বলে।

(x_1, y_1) $\quad \overrightarrow{AB} \quad$ (x_2, y_2)

$\overrightarrow{AB} = \{B(x,y) - A(x,y)\}$ – সাধারণ মান,

$\Rightarrow \{(x_2 - x_1)\hat{i} + (y_2 - y_1)\hat{j}\}$

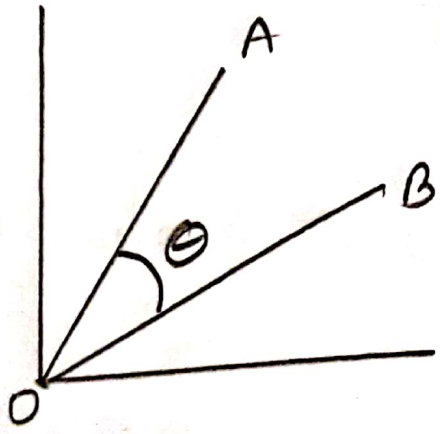
$\Rightarrow$ 2D

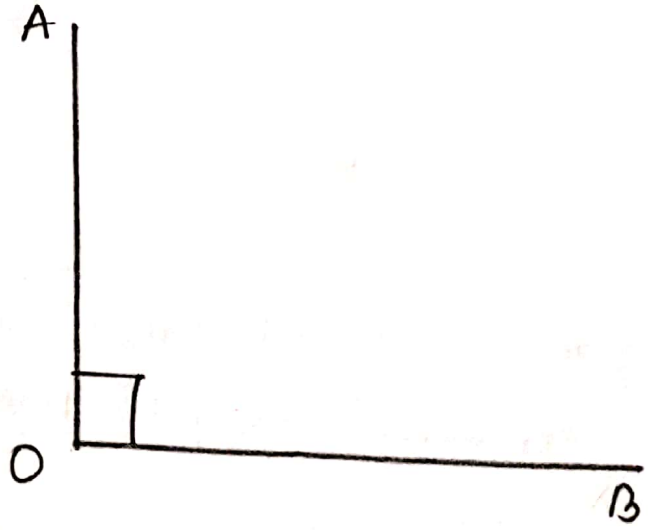
$\Rightarrow \{(x_2 - x_1)\hat{i} + (y_2 - y_1)\hat{j} + (z_2 - z_1)\hat{k}\}$

$= 3D$

$\overrightarrow{OA} \times \overrightarrow{OB} = OA \cdot OB \sin\theta \rightarrow$ সমান্তরিক

$\overrightarrow{OA} \cdot \overrightarrow{OB} = OA \cdot OB \cdot \cos\theta = 0 \rightarrow$ লম্ব।





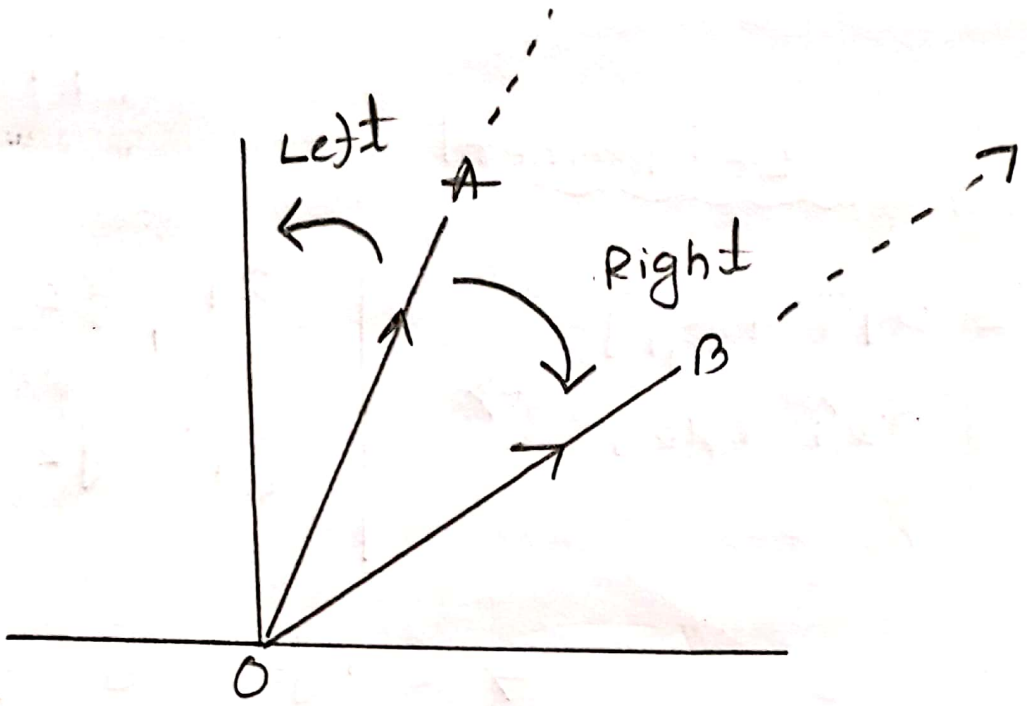
Jannatul Mawa

Determinent

$$OA = x_1\hat{i} + y_1\hat{j}$$
$$OB = x_2\hat{i} + y_2\hat{j} \rightarrow \begin{vmatrix} x_1 & x_2 \\ y_1 & y_2 \end{vmatrix}$$

$$\Rightarrow (x_1 y_2 - y_1 x_2)$$

- $\vec{OA} = x\hat{i} + y\hat{j}$
- $OA = \sqrt{x_1^2 + y_1^2}$
- $OB = \sqrt{x_2^2 + y_2^2}$
- $\cos 90^\circ = 0$
- ভেক্টর ঘূর্ণন অবস্থায় থাকে।



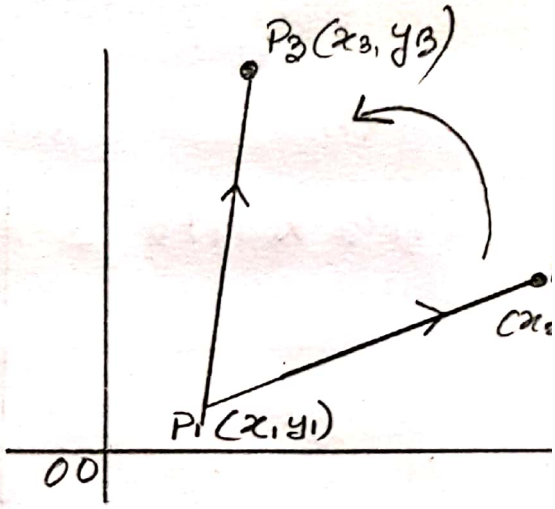
$[\vec{OA} \times \vec{OB}]\vec{OA}$ → clock wise → OB দিকে

$[\vec{OA} \times \vec{OB}]\vec{OA} > 0$ হয় তবে clock wise OB দিকে

$[\vec{OA} \times \vec{OB}]\vec{OA} < 0$ হয় তবে Anti clock OB দিকে

যা OB থেকে ঘুরে সরে যাবে,

$$\overrightarrow{P_1P_2} = (P_2 - P_1)$$
$$= (x_2 - x_1)\hat{i} + (y_2 - y_1)\hat{j}$$
$$\overrightarrow{P_1P_3} = (x_3 - x_1)\hat{i} + (y_3 - y_1)\hat{j}$$



Determinent:

$$P_1P_2 \times P_1P_3 = \begin{vmatrix} x_2 - x_1 & x_3 - x_1 \\ y_2 - y_1 & y_3 - y_1 \end{vmatrix} < 0$$

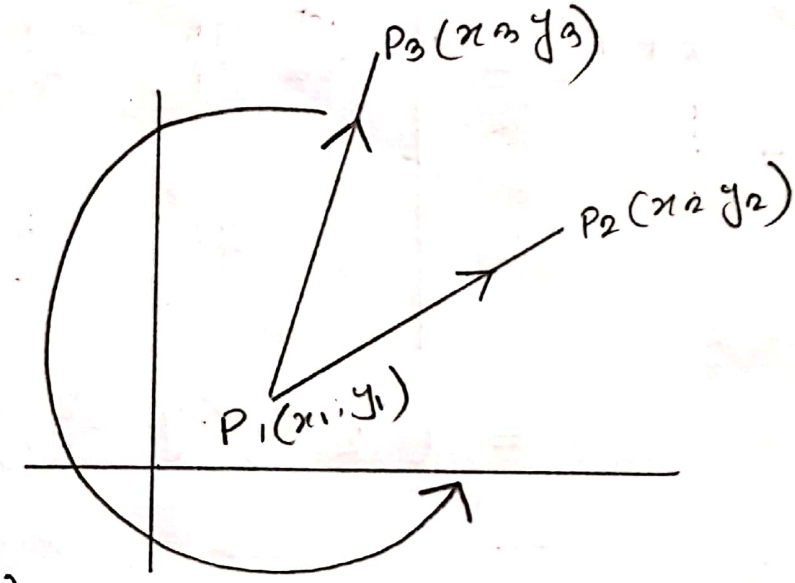
$$= (x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)$$

$$= x_2y_3 - x_2y_1 - x_1y_3 + x_1y_1 - x_3y_2 + x_3$$
$$- x_1y_2 + x_1y_1$$

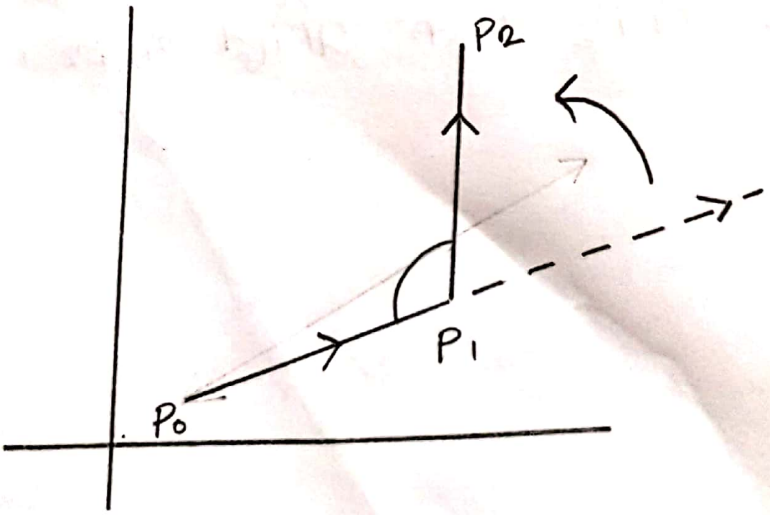
P_1P_2 anticlockwise P_1P_3 থেকে বামদিকে থাকছে, Left ship.

আবার,

$$P_1P_3 \times P_1P_2 = \begin{vmatrix} x_3 - x_1 & x_2 - x_1 \\ y_3 - y_1 & y_2 - y_1 \end{vmatrix} < 0$$



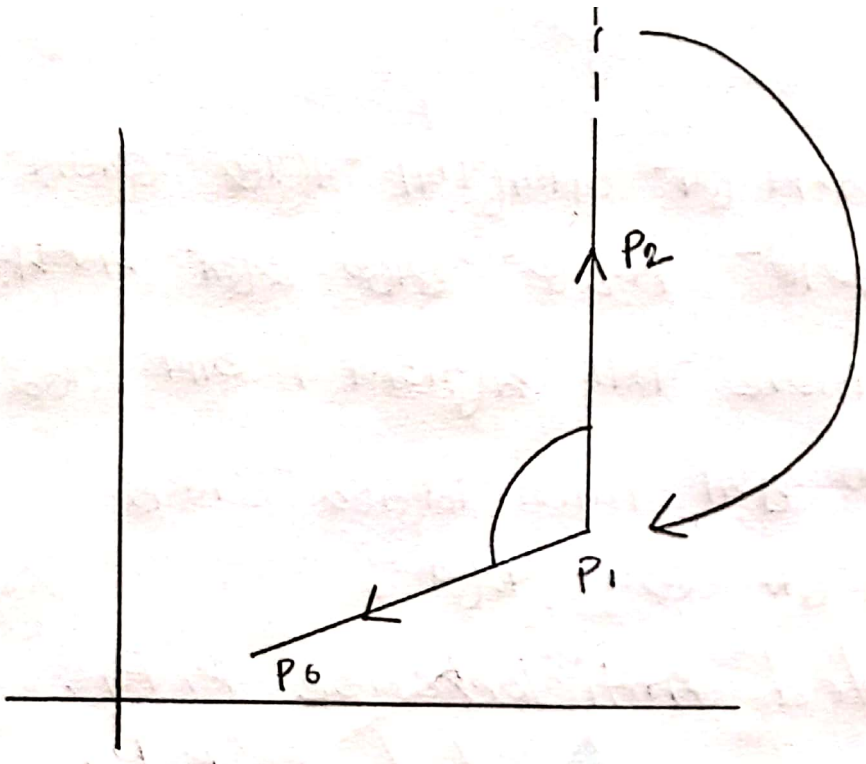
P_1P_3 Left shift হয়ে P_1P_2 এর দিকে যাবে,



$P_0P_1 \to P_1P_2$

Left ship.

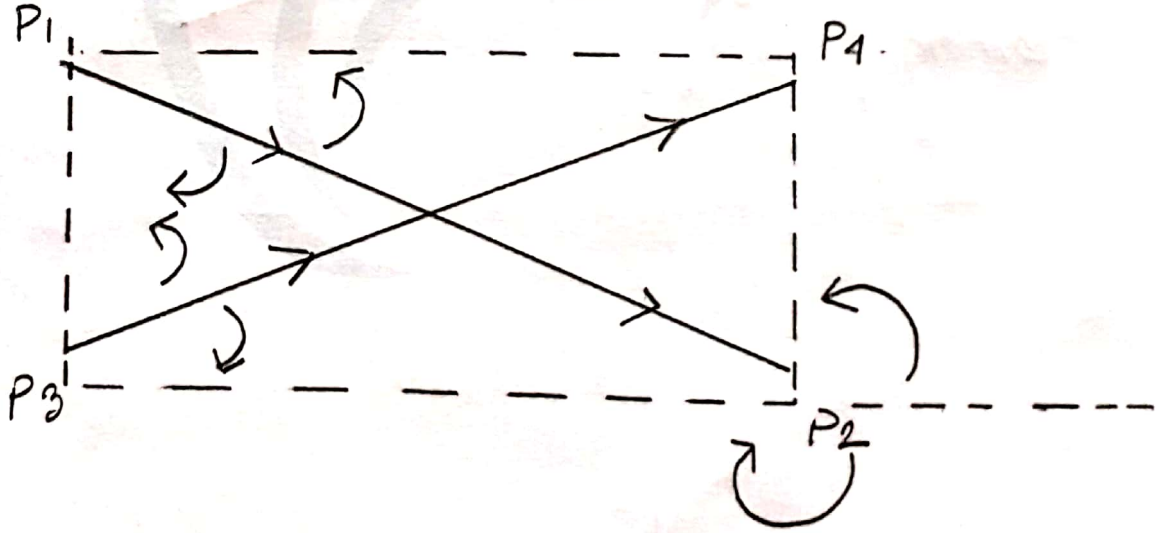
Anticlockwise.



$P_1P_2 \rightarrow P_0P_4$

Right Shift.

clockwise.



$P_1P_2 \rightarrow P_1P_3$

clockwise.

Jannatul Maoa

P_1 আপেক্ষে অপর যে সরলরেখা রয়েছে সেটার দুই প্রান্তের দুইটি বিন্দুতে যেতে হলে পরস্পর বিপরীতমুখী Move করা প্রয়োজনে। আর সেহেতু টিকে বিপরীত মুখী turn নিয়েছে সেহেতু P_1, P_2, P_3, P_4 কে চেক করে,

P_1P_2 directed Anticlockwise P_1P_4.

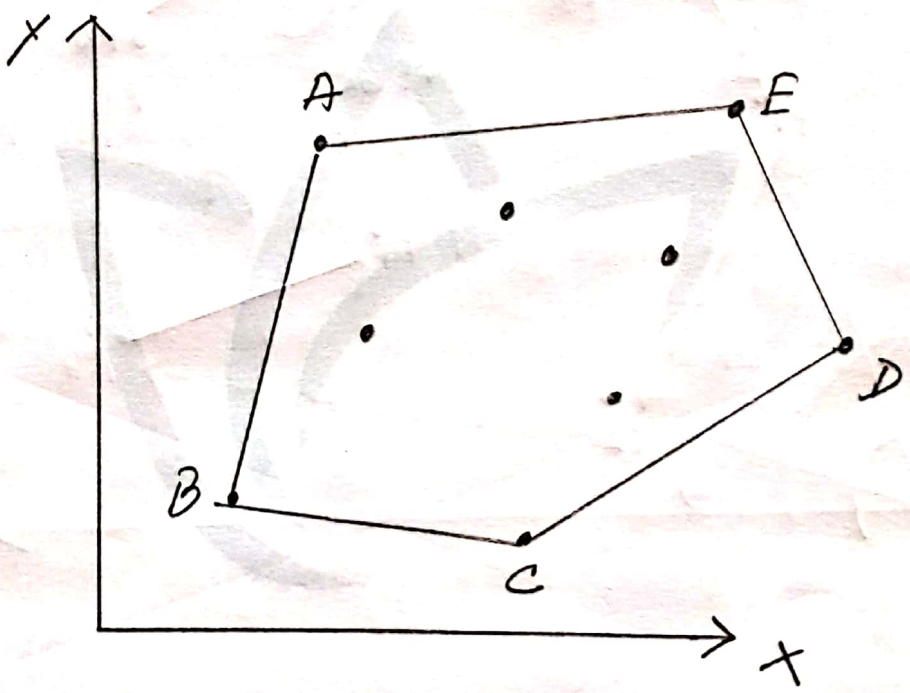
P_1P_2 clockwise directed to P_1P_3

So those direction is opposite to each other. They are not intersect each other.

Convex Hull

Convex 2D plane এর জন্য কাজ করে।

Convex Hull: 2D plane এর ভিতরের বিন্দু গুলো সমন্বভাবে Connect করতে এবং চারবিন্দু এক তেমাত্রে বা এক বহুভুজের ভিতর আবদ্ধ করে।



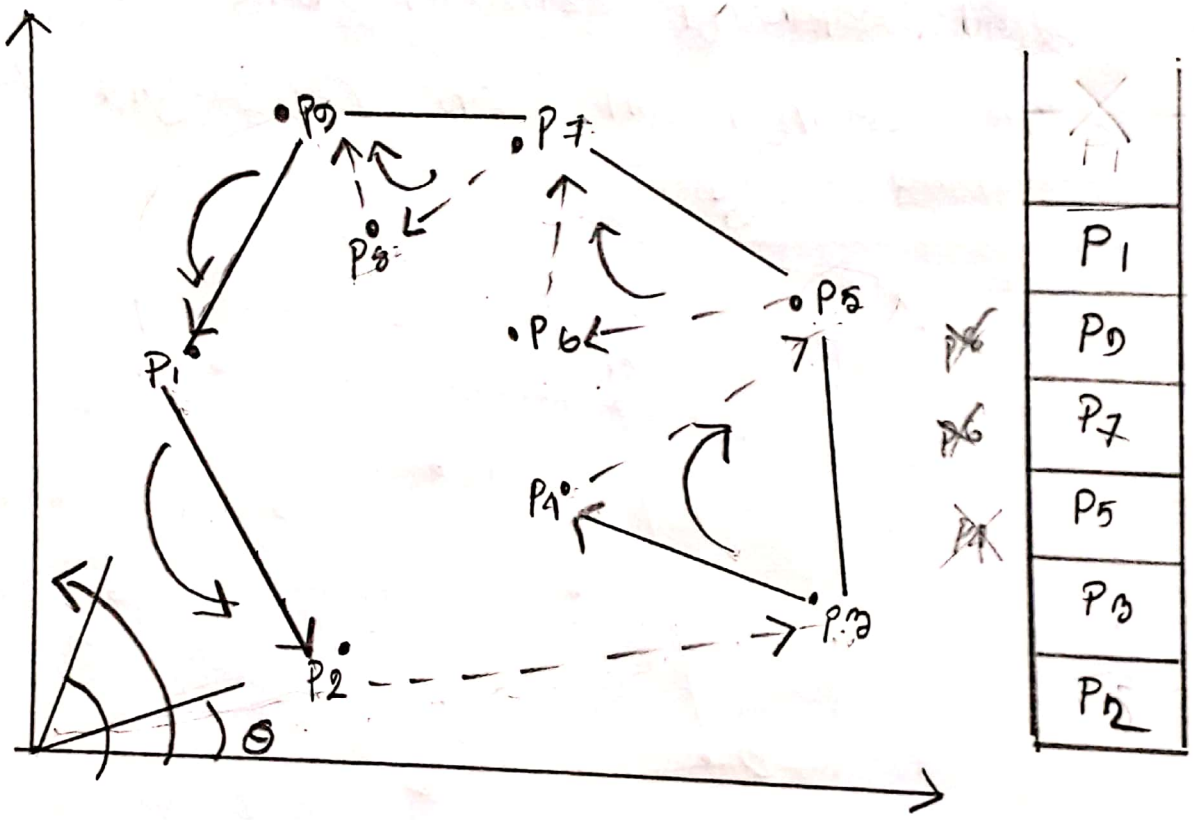
Jannatul Maoa

Graham's Scan Algo

*

Right shift ২বার হলে
তাহলে POP মানে।

(Polar position)



By following polar Angle Array

P_2 P_3 P_4 P_5 P_6 P_7 P_8 P_9 P_1

Greatest Common Devision

Euledia Process: (9, 24)

Divisor ↓

Deviding 9 | 24 | 2
18
6
↑ Modaler

9 | 24 | 2
18
6 | 9 | 1
6
3 | 6 | 2
6
X

(code) : -- gcd (a, b);

Jannatul Maoua

Backtracking

Backtracking

↓

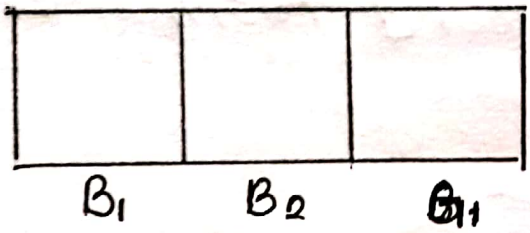
(Brute force approach or Branch & Bound)

↓ ↓

DFS BFS.

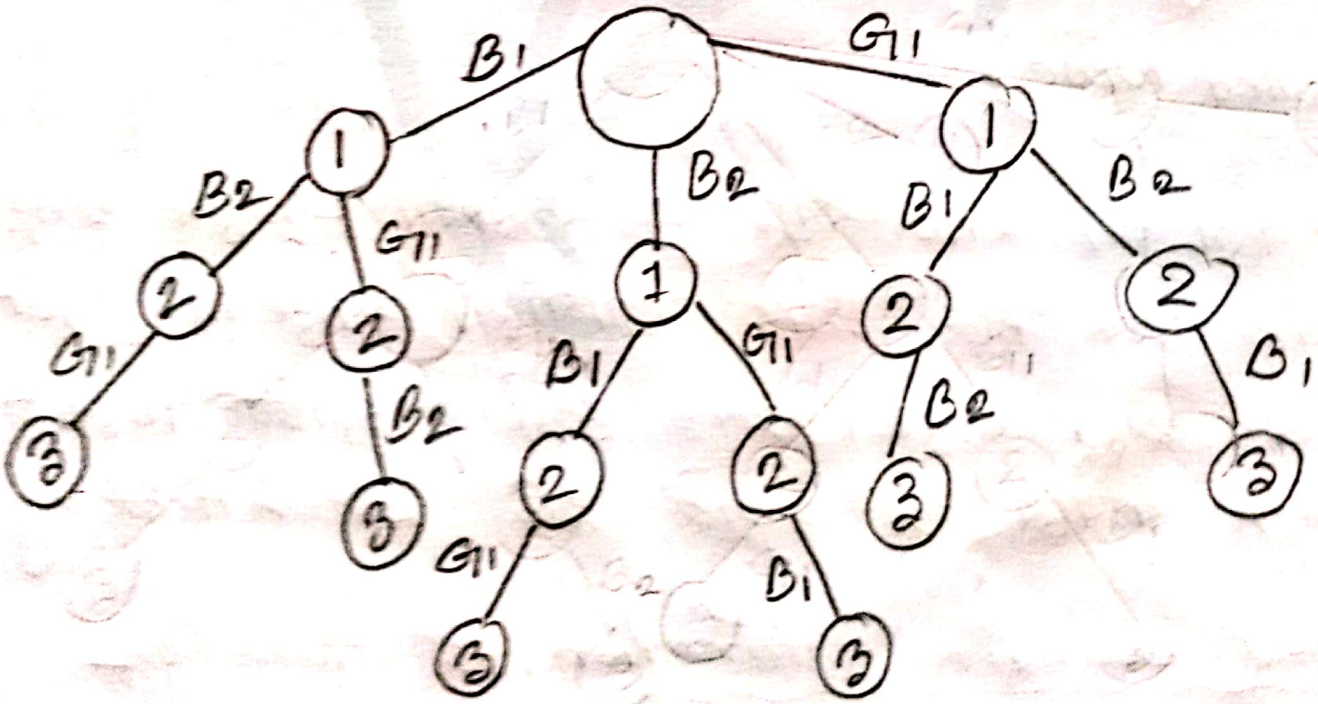
- যদি কোনো Problem এ অপশনগুলো Solution থাকে এবং সবগুলো Solution নিয়ে কাজ করতে হয় সেক্ষেত্রে Backtracking use করা হয়।
- State Space tree : সম্ভাব্য সকল ফলাফল নিয়ে আকা tree.
- Backtracking depends on state Space tree.

Jannatul Maoua

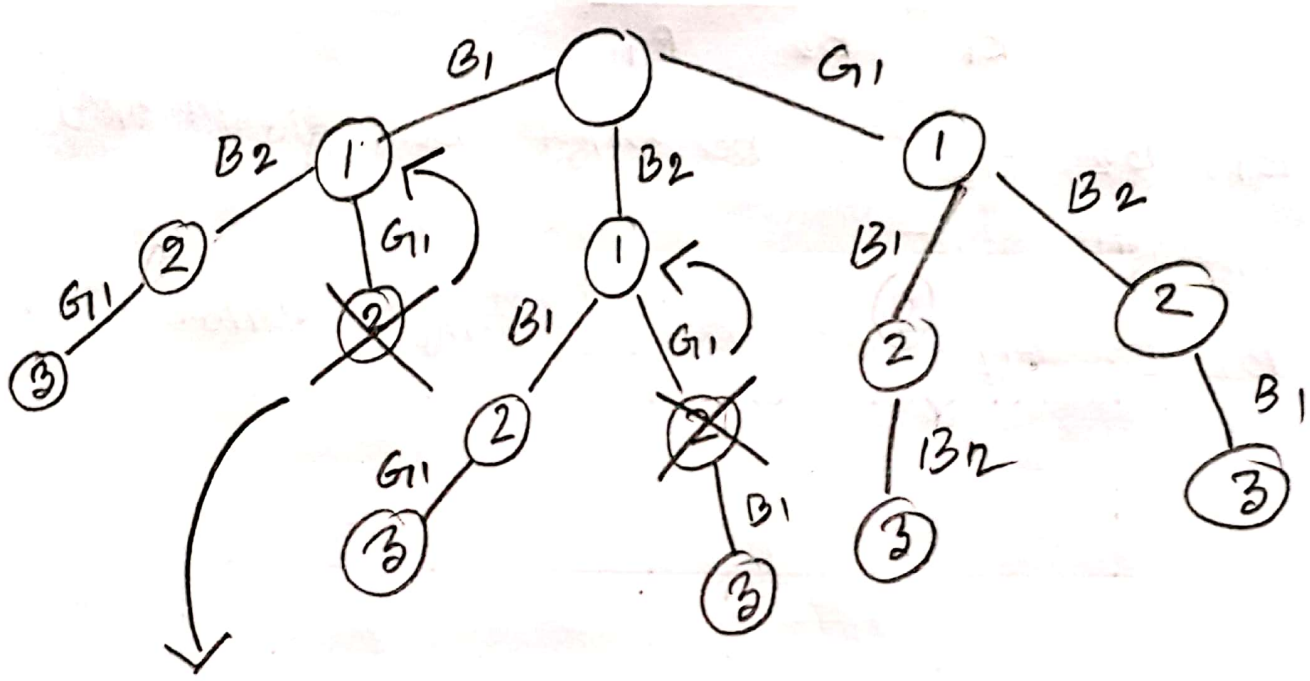


B_1, B_2, G_1 -কে কত উপায়ে Box গুলোর মধ্যে সাজানো যায়।

B_1, B_2, G_1 -কে $3! = 6$ উপায়ে সাজানো যায়,



* যদি বলা হয় G1 Second Position এ বসাবে না।



যদি- বলে G1 2NO বক্স এ বসাবে না তাহলে 2নং- Node এর পর আর আগানো যাবে না। সেটা Cross হয়ে যাবে। এবং সেটা উল্টো দিকে- Back করবে এটি তাই এটি Breaktracking.

Killed bounding function: এ function killed creat করে node diteet করে, them Node-কে- kill করে ফেলে।

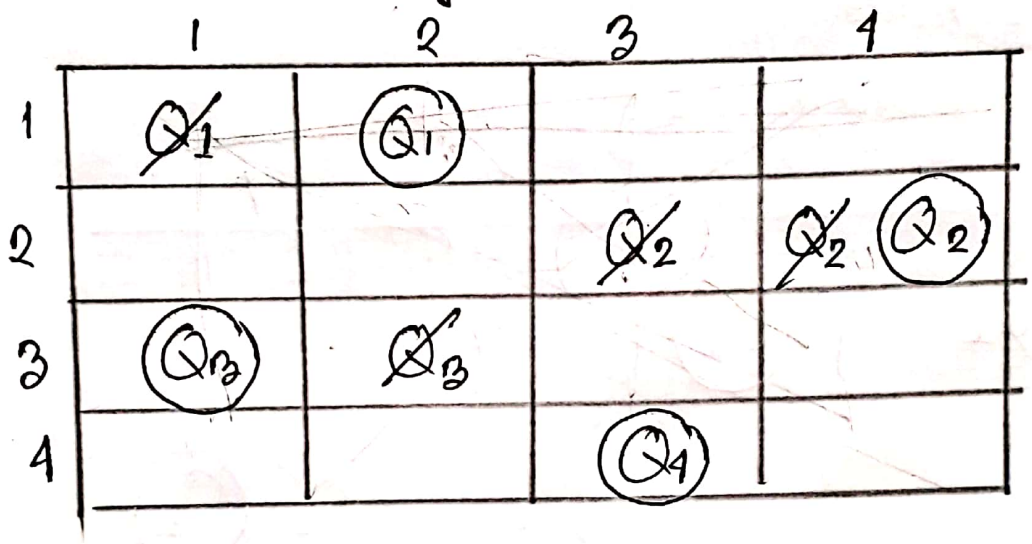
N-Queen's Problem

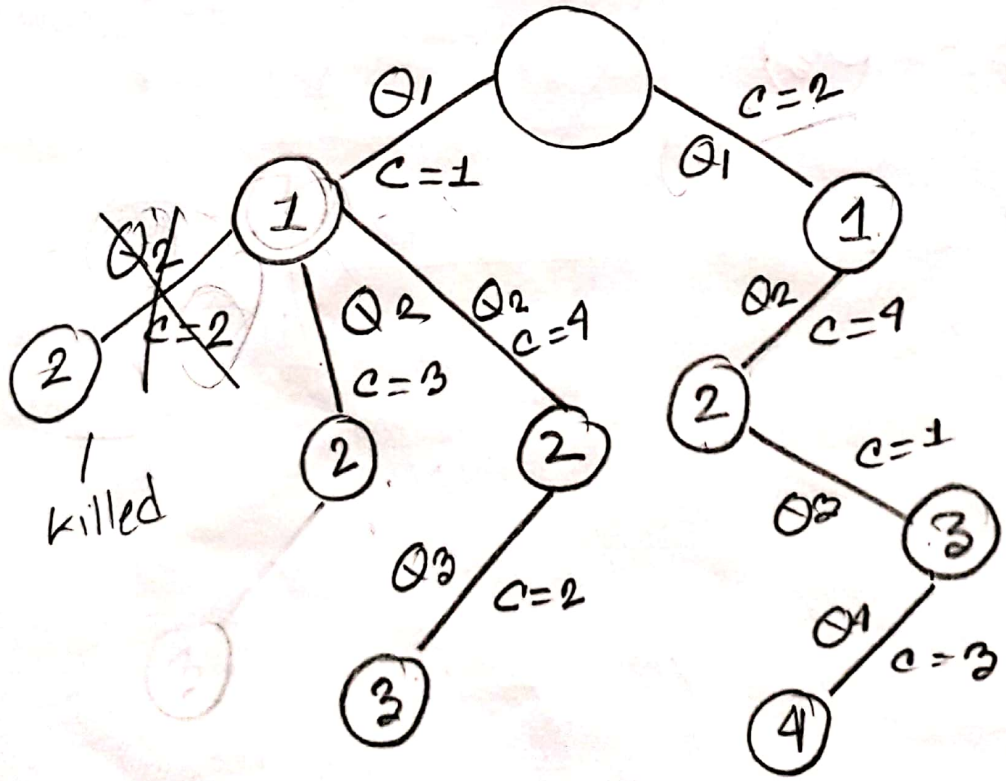
Jannatul Maowa

(Backtracking)

$N = 4$ (Q_1, Q_2, Q_3, Q_4)

level = row ; edge = column ;





Soluation

Jannatul Maoa

2	4	1	3
1	2	3	4

∴ Only n Queens এর জন্য সমাধান।
Another Soluation.

3	1	4	2
1	2	3	4

Robin kamp - Algo

Jannatul Maasa

Robin - kamp - Algo

↓

(Sub-string বের করার জন্য use করা হয়) → (Char গুলো ধারাবাহিক sequence হয়ে থাকতে হবে)

↓

(Pattern detector)

⊞ Robin kamp Algo programe এ TLE এড়ানোর জন্য এর Update Version Hashing use করা হয়, Hashing function - এর সাহায্যে Program - কে - Optimal করা হয়,

S_1 = c c a b c d e ?

S_2 = a b e

1 + 2 + 3

= 6

S_1 = c c a b c d e ? y

S_2 = a b y X

where, a = 1

b = 2

c = 3

d = 4

e = 5

f = 6

y = 26

[এভাবে চেষ্টা করতে হবে কোন - তিনটি String এর যোগফল 6, এবং তাদের Search করতে হবে এগুলো Sub String আকারে - আছে কি না].

So, এই Process টাকে General Solution বলা হয় যার Time Comixity = $O(n^2)$, তাই T.C কমানোর জন্য এর Solution হিসেবে Hashing function use করা হয় যার

$TC = O(n)$. [গুণের ক্ষেত্রে = $O(1)$
3টি -কোনার Read করার = $O(n)$]

Hashing function :

$P[i] \times base^{m-1} + P[i+1] \times base^{m-2} + P[i+2] \times base^{m-3}$

⇒ hash code.

$TC = O(n)$.

$S_1 = c\ c\ a\ b\ c\ d\ e\ f\ g$

$S_2 = a\ b\ c$

$1 \times 7^2 + 2 \times 7^1 + 3 \times 7^0$ → base. → Hashing function.

$= 49 + 14 + 3$

$= 66$ → Hash Number.

[গুণের operation Read $O(1)$
3টিকে কোনার Read $O(n)$]

Jannatul Macon

Then, c c d

$3 \times 7^2 + 3 \times 7^1 + 4 \times 7^0$

$= 147 + 21 + 4$

$= 172$ ✗ 169

Then c a b . X

Then a b c ✓

এখন sequence অনুসারে Search করবে করে Hashing Num বের করে চেক করবে Sr এর সাথে Mateh করে কিনা, Numbere Mateh করলে ফলাফল পাওয়া যাবে,

01 knapsack Problem (Non-Fractional)

↓

using Dynamic Programming DP solution.

Suppose;

m (capacity) $= 8$ kg

and $n = 4$

$P = \{1, 2, 5, 6\}$ $W = \{2, 3, 4, 5\}$

↓ Profit ↓ Weight.

P_i (Colum Value)	ω_i (Colum Num)
$P_1 = 1$	2
$P_2 = 2$	3
$P_3 = 5$	4
$P_4 = 6$	5

Sort depend on profit.

$8 - 6 = 2$

$2 - 2 = 0$

Profit \ Weight	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	(0)	1	1	1	1	1	1	1
2	0	0	1	(2)	2	(3)	3	3	3
3	0	0	1	2	(5)	5	(6)	(7)	7
4	0	0	1	2	5	(6)	6	7	(8)

$\sum P_i$ - Profit.

P_1	P_2	P_3	P_4
0	1	0	1

P_2, P_4

$\sum P_i = 8$ TK

$\sum W_i = 8$ kg

Jannatul Mawa

Formula:

$$[V[i,w] = \max\{V[i-1,w], V[i-1, w-W[i]] + P[i]\}]$$

for the Generical / Normal Symtom

Time complixeity = $O(2^m)$

$= O(2^{capaeity})$

for the Dynampie Programing (DP)

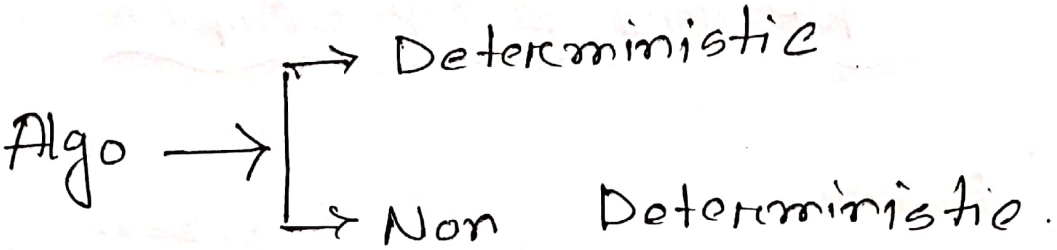
Time Compixity = $O(m^2)$

$= O(capaeity^2)$

NP-Hard & NP-Complete

(Desire Polynominal Time	Exponential
Linear Search (n)	0/1 knapsack 2^n
Binary " (logn)	TSP 2^n
Insertion Sort (n^2)	Circuit 2^n
Merge Sort (nlogn)	Hamiltonial cycle 2^n
MCM (n^3)	
↑ P	

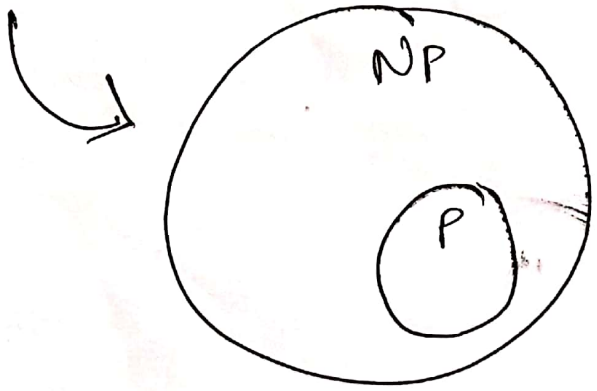
Jannatul Maowa



P → set of those Deterministic Algo which take polynomial time.

NP → set of these Non Deterministic Algo which polynomial time.

All P is a subset of NP.



(NP - P Diagram)

CNF Satisfiability

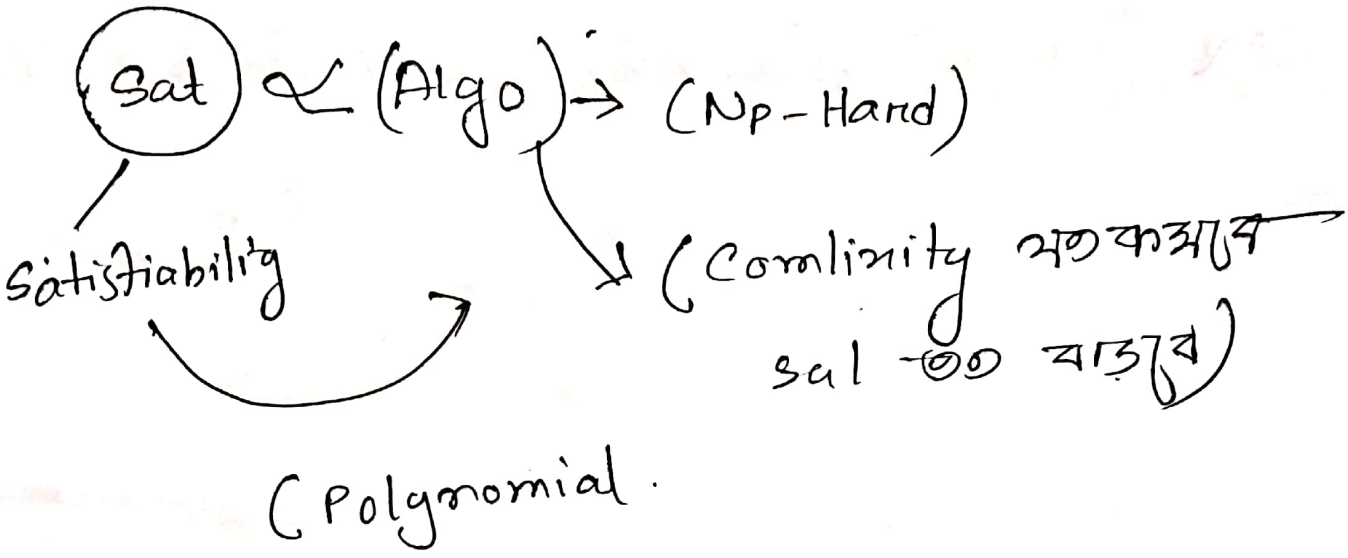
$x_i = \{x_1, x_2, x_3 \ldots\ldots\ldots x_n\}$

CNF formula $= (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee \bar{x}_3)$

$(2^n \rightarrow n^2)$

Reduction

N/P Hard



(Polynomial.

Exponential ⟶ Polynomial

Time Complexity — Jannatul Macca

BFS, DFS = $O(n^2)$

n Queens = $O(n!)$ $O(n^2)$

Prims, kruskal, dijkstra = $O(n^2)$

(nlogn)

floyd warshal = $O(n^3)$.

Bellman Normal = $O(n^2)$

complete = $O(n^3)$

Rabin karp = $O(n)$

0/1 knpsak = Normal → $O(2^{\text{complinity}})$

DD = $O(n^2)$

line segment = $O(\log n)$

Activity Log = $O(n)$

Graph / tree traversal

Jannatul Maoa

1. BFS (Breath first Search) → Queue.
2. DFS (Depth first Search) → Stack.

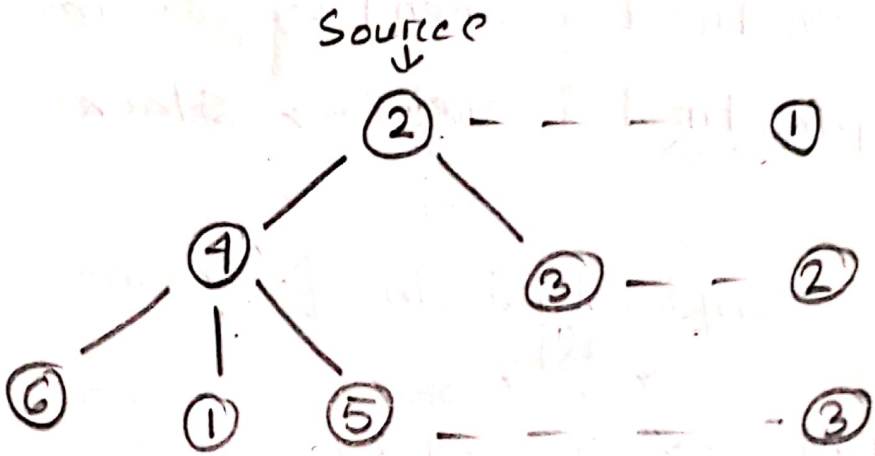
The time complexity both of them:

$$O(E+V) \Rightarrow O(V+V) \quad [E = V]$$
$$\Rightarrow O(2V)$$
$$= O(n)$$

[E = Edges
V = vertex or Node

for worst case $O(n^2)$

1. BFS (Breath first Search) / (Level order Traversal)



Queue:

X	X	X	X	X	X
2	4	3	6	1	5

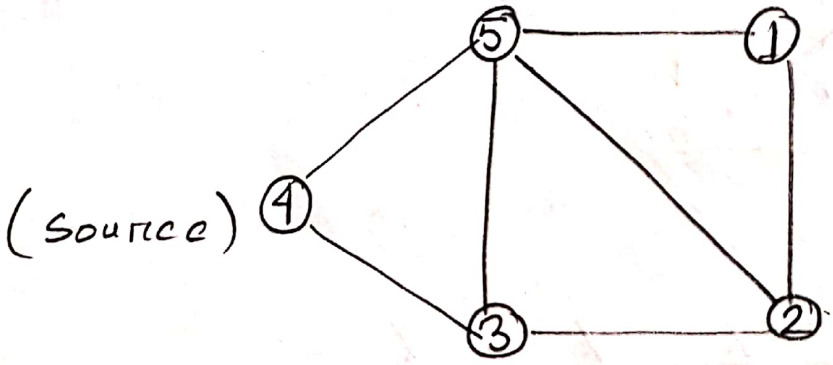
visit array:

0	1	2	3	4	5
0	1	1	1	1	1

BFS Output: 2, 4, 3, 6, 1, 5

BFS for Graph

Jannatul Maowa



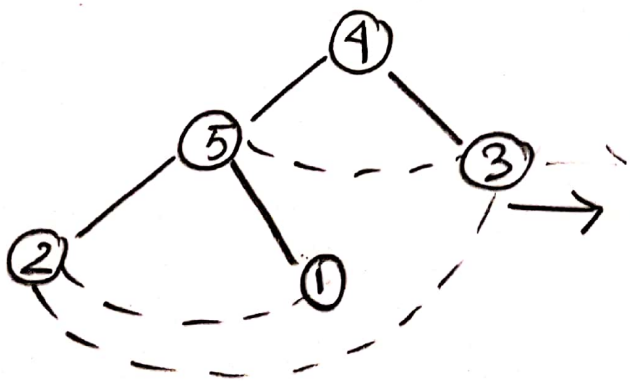
Queue:

X	X	X	X	X
4	5	3	2	1

Visit array:

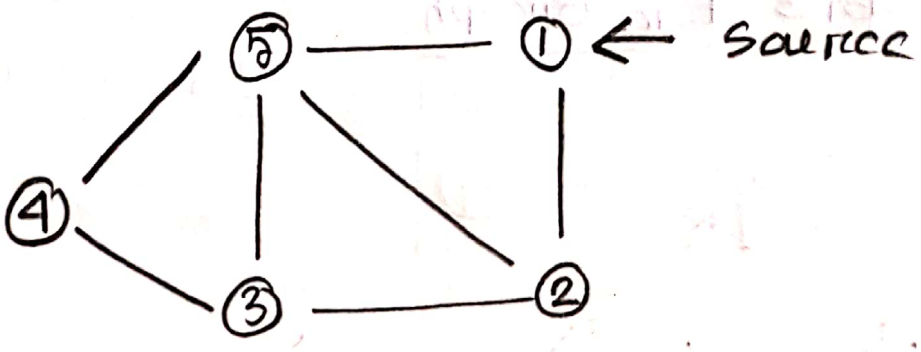
0	1	2	3	4	5
0	1	1	1	1	1

Output: 4, 5, 3, 2, 1

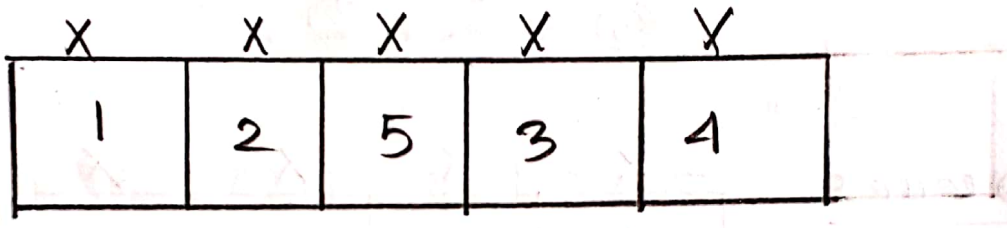


Jannatul Mawa

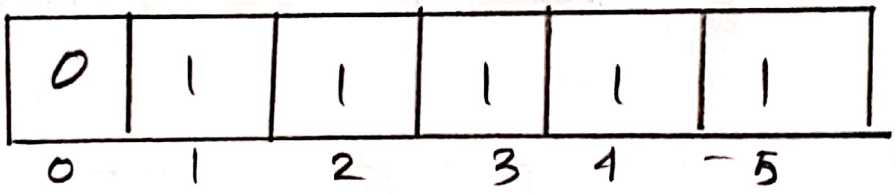
Ex:



Queue:



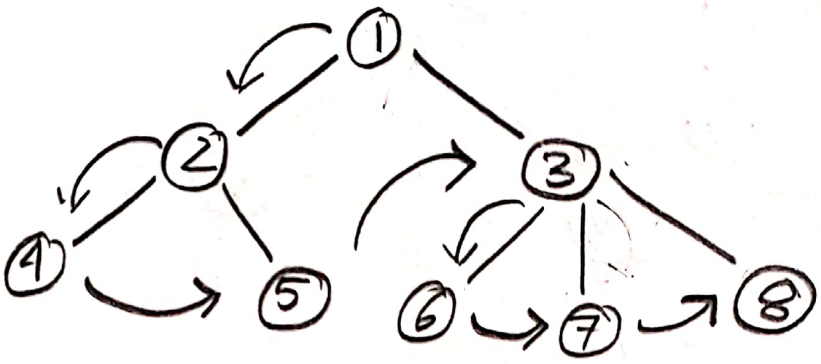
Visit Array:



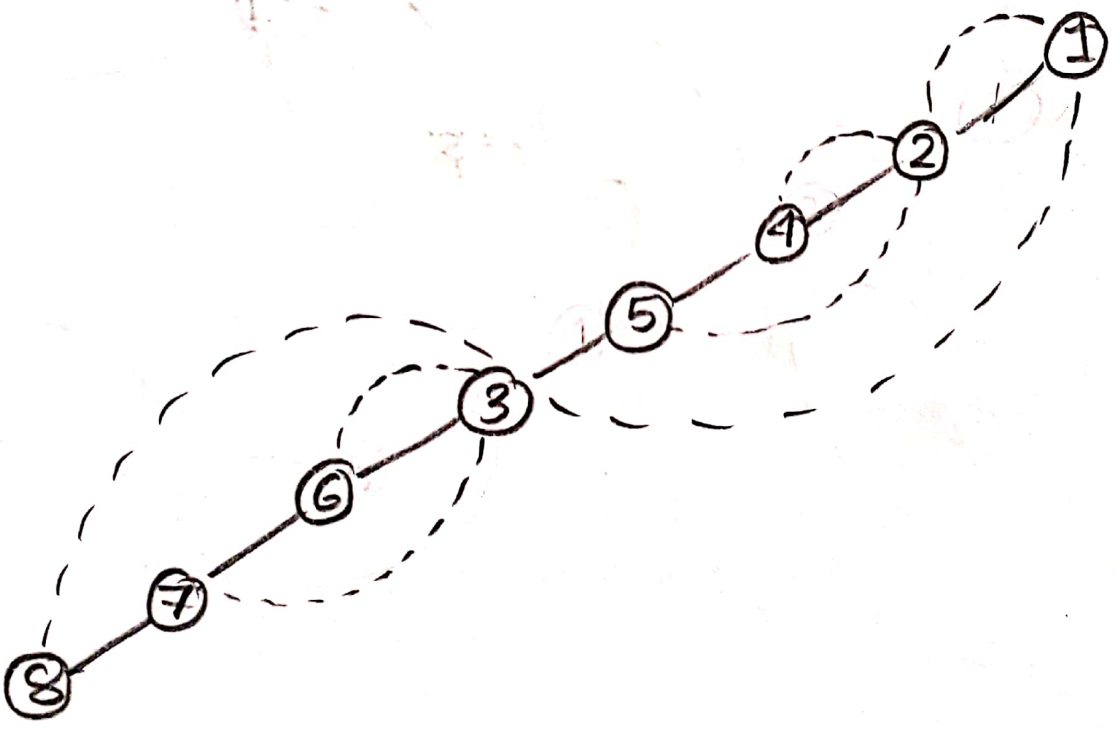
Output: 1 2 5 3 4

DFS (Depth first Search) → Stack

Jannatul Macoa

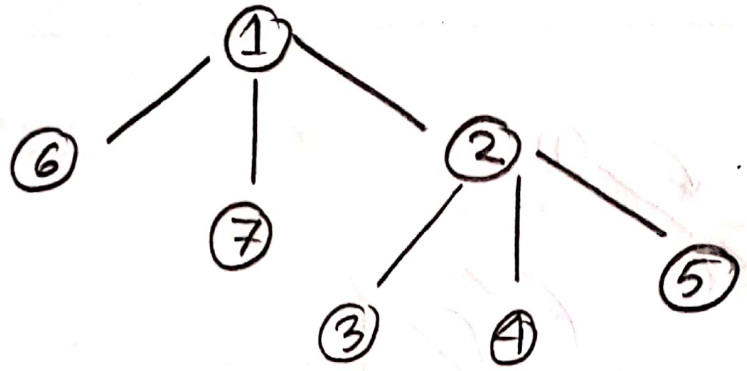


DFS : 1 2 4 5 3 6 7 8

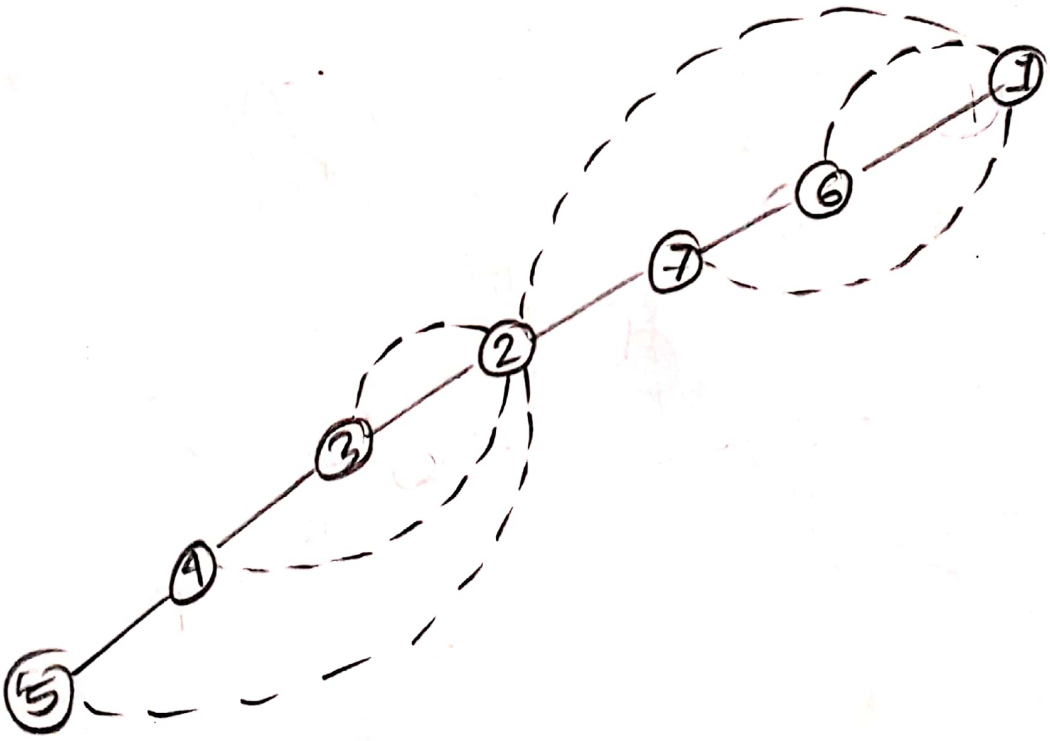


Ex:

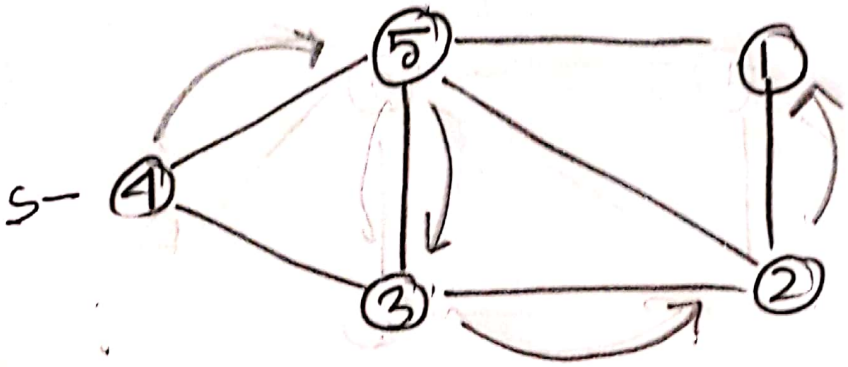
Jannatul Maoua



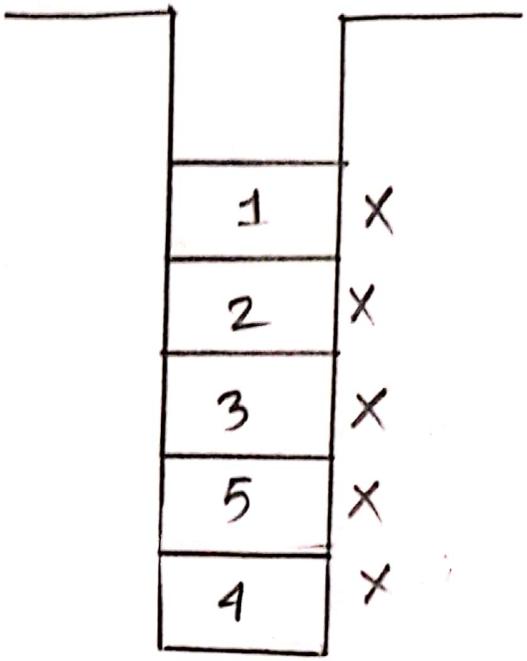
DFS: 1 6 7 2 3 4 5



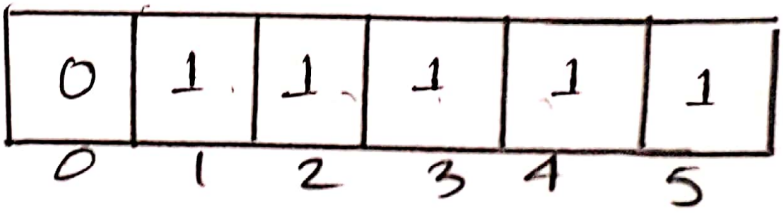
DFS - for Graph



Stack



Visit array:



Output: 4 3 2 1 5